



UNIVERZITET U NOVOM SADU
FAKULTET TEHNIČKIH NAUKA
U NOVOM SADU



PROJEKAT

iz Projektovanja složenih digitalnih sistema

TEMA PROJEKTA:

ESL modelovanje i hardverska akceleracija algoritma senzorne fuzije IMU podataka
(Madgwick IMU filter)

PROJEKAT IZRADILI:

Jelena Klisurić, EE62/2021
Aleksandra Vukašinović, EE2/2022

MENTOR:

MSc Jana Janković

U Novom Sadu, 2026.

Sadržaj

1. Opis algoritma koji se akcelerira.....	3
2. Paralelizacija operacija.....	4
3. Interfejs dizajna i njegovo okruženje.....	5
4. Pipeline arhitektura sekcije 678.....	6
5. Blok dijagram komponenti.....	10
5.1 Detaljna struktura sekcije 5.....	11
6. Pakovanje IP jezgra.....	12
7. Analiza integrisanog sistema nakon sinteze i implementacije.....	14
7.1 Izgled integrisanog sistema.....	14
7.2 Utrošeni resursi.....	15
7.3 Kritična putanja i maksimalna frekvencija.....	16
7.4 Propusna moć i kašnjenje.....	18
8. Poređenje sa rezultatima ESL projekta.....	19
8.1 Diskusija razlika.....	19
9. Testiranje rada u Vitis-u.....	20
9.1 Tok obrade jednog sample-a.....	20
9.2 Format CSV ispisa.....	21
9.3 Verifikacija ispravnosti.....	21
9.4 Poređenje sa hardverskim modelom iz virtuelne platforme.....	23
9.5 Tačnost i akumulacija greške.....	25
9.6 ILA kao dodatna provera.....	25
10. Problemi tokom razvoja i njihova rešenja.....	26
10.1 Pogrešna pretpostavka o latenciji FP IP jezgara.....	26
10.2 Sekvencijalna konverzija float u fixed kao usko grlo.....	26
10.3 Vremensko usklađivanje provere dot proizvoda.....	27
10.4 Sinhronizacija nakon Quake inverse sqrt-a.....	27
10.5 Agresivna optimizacija Vivado sintetizatora.....	27
10.6 Q-format manipulacija - explicit shift_right + resize.....	28
10.7 Debug infrastruktura kroz pipeline.....	29
10.8 Upotreba jednog fp_mul IP jezgra.....	29
10.9 Verifikacija ispravnosti kroz mixed simulation.....	30
11. Literatura.....	31

1. Opis algoritma koji se akcelerira

Algoritam koji je akcelerisan je AHRS (Attitude and Heading Reference System) algoritam koji služi za procenu orijentacije objekta u prostoru pomoću kvaterniona. Implementirane su sekcije 5 do 8, koje zajedno čine jedan korak integracije kvaterniona.

Sekcija 5 se bavi kompenzacijom greške akcelerometra. Na osnovu izmerenih vrednosti ubrzanja i vektora gravitacije koji dolazi iz softverskog dela sistema, računa se korekcioni vektor hfb . Najpre se normalizuje vektor ubrzanja korišćenjem algoritma brzog inverznog kvadratnog korena poznatog kao *Quake fast inverse square root*, sa magičnom konstantom 0x5F1F1412 i jednom iteracijom *Newton-Raphson korekcije*. Nakon toga se računa vektorski proizvod normalizovanog ubrzanja i vektora gravitacije. Ukoliko je skalarni proizvod ta dva vektora negativan, što znači da su okrenuti u suprotnim smerovima, vektorski proizvod se dodatno normalizuje. Rezultat ove sekcije su tri vrednosti, $hfbx$, $hfbz$ i $hfbz$ u formatu Q2.24.

Sekcija 6 pretvara vrednosti žiroskopa iz deg/s u polovinu ugaone brzine izraženu u rad/s. Ulazni format je Q9.7 (signed 16-bit), a izlazni Q2.20 (22-bit).

Sekcija 7 primenjuje korekciju akcelerometra na merenje žiroskopa računajući korigovanu polu-ugaonu brzinu po formuli $adjHalfGyro = halfGyro + hfb \times gain$, gde gain predstavlja faktor koji se postepeno povećava pri pokretanju sistema.

Sekcija 8 integriše kvaternion Ojlerovom metodom. Najpre se računa derivacija kvaterniona kao vektorski proizvod trenutnog kvaterniona i korigovane ugaone brzine, što zahteva 12 množenja. Zatim se kvaternion ažurira po formuli $q_{new} = q + dq \times dt$. I ulazni i izlazni kvaternion su u formatu Q2.16 (signed 18-bit).

2. Paralelizacija operacija

U originalnom algoritmu nije bilo eksplicitnih petlji koje bi trebalo odmotavati. Međutim, sekcija 5 je prvobitno bila implementirana sekvencijalno, tj. svaka floating-point operacija čekala je na rezultat prethodne. Ovakav pristup je doveo do latencije od oko 180 taktova, što je bilo neprihvatljivo za ciljanu radnu frekvenciju.

Da bi se latencija smanjila, izvršena je paralelizacija svih operacija koje su međusobno nezavisne. Umesto jednog floating-point množača, instancirane su tri nezavisne paralelne instance istog Xilinx Floating Point v7.1 IP jezgra `fp_mul` (`U_MUL`, `U_MUL2` i `U_MUL3`) koje mogu da rade istovremeno. Sve tri instance dele identičnu konfiguraciju latencija od 4 takta i AXI-Stream interfejs. Na taj način, operacije poput računanja $ax \times ax$, $ay \times ay$ i $az \times az$ više ne čekaju jedna na drugu, već se izvršavaju paralelno u jednom koraku. Isti princip primenjen je na normalizaciju vektora ubrzanja, računanje vektorskog proizvoda i skaliranje izlaza.

Zahvaljujući ovoj paralelizaciji, latencija sekcije 5 je značajno smanjena u odnosu na prvobitnu sekvencijalnu implementaciju. Konkretno brojne vrednosti latencije izmerene su u simulaciji pomoću internog cycle brojača (`r_cycle_cnt`) ugrađenog u FSM sekcije 5.

U sekciji 678, svih 12 množenja potrebnih za računanje derivacije kvaterniona mapirana su na DSP ćelije FPGA-a i izvršavaju se paralelno u jednom taktu pipeline-a.

3. Interfejs dizajna i njegovo okruženje

Top-level entitet je *ahrs_sec5to8*, koji strukturno povezuje *ahrs_sec5* i *ahrs_sec678*. Interfejs je sledeći:

Port	Tip	Format	Opis
clk	ulaz	std_logic	Taktni signal
reset	ulaz	std_logic	Asinhroni reset
start	ulaz	std_logic	Pokretanje izračunavanja
ax_i, ay_i, az_i	ulaz	IEEE 754 float32	Vrednosti akcelometra
hgx_i, hgy_i, hgz_i	ulaz	IEEE 754 float32	Poluvektor gravitacije (sa SW sekcije 4)
gx_i, gy_i, gz_i	ulaz	signed Q9.7 (16-bit)	Vrednosti žiroskopa
gain_i	ulaz	unsigned Q4.16 (20-bit)	Gain faktor
qw_i, qx_i, qy_i, qz_i	ulaz	signed Q2.16 (18-bit)	Trenutni kvaternion
dt_i	ulaz	unsigned Q0.20 (20-bit)	Vremenski korak
ready	izlaz	std_logic	Signal da je rezultat spreman
qw_o, qx_o, qy_o, qz_o	izlaz	signed Q2.16 (18-bit)	Novi kvaternion

Okruženje dizajna: *ahrs_sec5to8* je instanciran unutar *top_zybo* entiteta na Zybo Z7-10 ploči (Xilinx Zynq xc7z010clg400-1). Taktni signal dolazi iz Clocking Wizard IP jezgra koje generiše 70MHz iz ulaznog 125MHz takta. Izlazi *qw_o* i *qx_o* (bit 17, predznak) su prikazani na LED diodama.

4. Pipeline arhitektura sekcije 678

Sekcija 678 implementirana je kao četvorostepeni pipeline bez state machine-a. Svaki takt donosi novi rezultat na izlaz uz fiksnu latenciju od četiri takta od momenta postavljanja start signala.

Stage	Naziv	Operacija
Stage 1	Sekcija 6	$\text{halfGyro} = \text{gx} \times (\pi/360)$, konverzija Q9.7 $\rightarrow$ Q2.20
Stage 2	Sekcija 7	$\text{adjHalfGyro} = \text{halfGyro} + \text{hfb} \times \text{gain}$
Stage 3a	Sekcija 8a — množenja	12 paralelnih množenja $q \times \text{adjHalfGyro}$ na DSP48E1
Stage 3b	Sekcija 8a — akumulacije	Računanje sw , sx , sy , sz i slicing rezultata
Stage 4	Sekcija 8b	$q_{\text{new}} = q + \text{dq} \times \text{dt}$, postavljanje izlaza

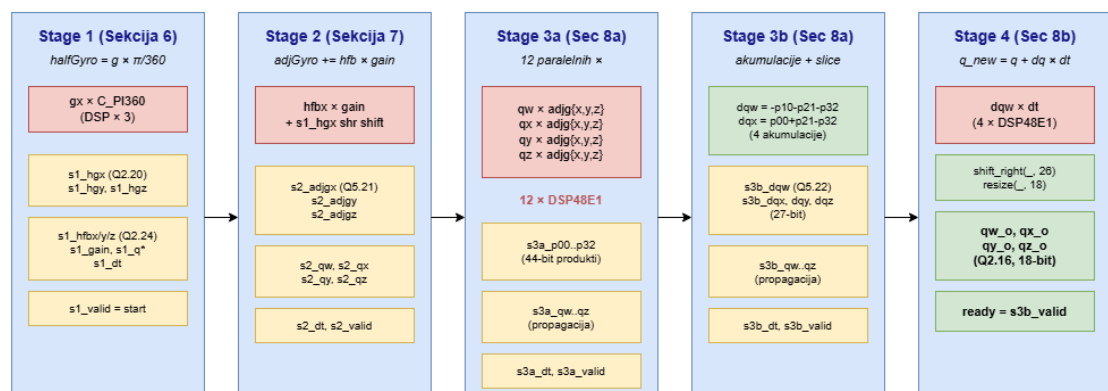
Originalni Stage 3 je u jednom taktu obavljao i sva množenja i akumulacije, što je činilo najduži kritični put u dizajnu i ograničavalo maksimalnu radnu frekvenciju. Razbijanjem na Stage 3a i Stage 3b kritični put je prepolovljen i radna frekvencija je povećana na 70 MHz.

Ulazi:

gx_i , gy_i , gz_i (signed Q9.7, 16-bit), hfbx_i , hfby_i , hfbz_i (signed Q2.24, 26-bit), gain_i (unsigned Q4.16, 20-bit), qw_i , qx_i , qy_i , qz_i (signed Q2.16, 18-bit), dt_i (unsigned Q0.20, 20-bit), start (std_logic)

Izlazi:

qw_o , qx_o , qy_o , qz_o , ready



Slika 1. Pipeline sekcija 6, 7 i 8

Sekcija 5 je implementirana kao state machine sa dva stanja: **S_IDLE** i **S_RUN**. U stanju *S_IDLE* dizajn čeka na signal *start*, a prelaskom u *S_RUN* prolazi kroz sekvencu od 33 koraka u kojima svaki floating-point korak čeka latenciju IP jezgra (3 takta za množenje, 4 takta za sabiranje/oduzimanje).

Sekvenca koraka odgovara matematičkom toku algoritma fast inverse square root i vektorskih operacija:

- Koraci 1-3: paralelno množenje $ax \times ax$, $ay \times ay$, $az \times az$ na tri instance fp_mul.
- Koraci 4-5: sabiranje (floating_point0) - $accelMagSq = ax^2 + ay^2 + az^2$
- Koraci 6-12: fast inverse sqrt - manipulacija bita + Newton-Raphson korekcija
- Koraci 13-15: paralelna normalizacija nx , ny , nz
- Koraci 16-22: vektorski proizvod $cross = n \times hg$
- Koraci 23-25: skalarni proizvod $dot = n \cdot hg$
- Grananje na osnovu $dot < 0$: -

Ako je $dot \geq 0$: direktno na konverziju u Q2.24 (ukupno 58 taktova)

Ako je $dot < 0$: dodatna normalizacija cross vektora (28 dodatnih taktova,

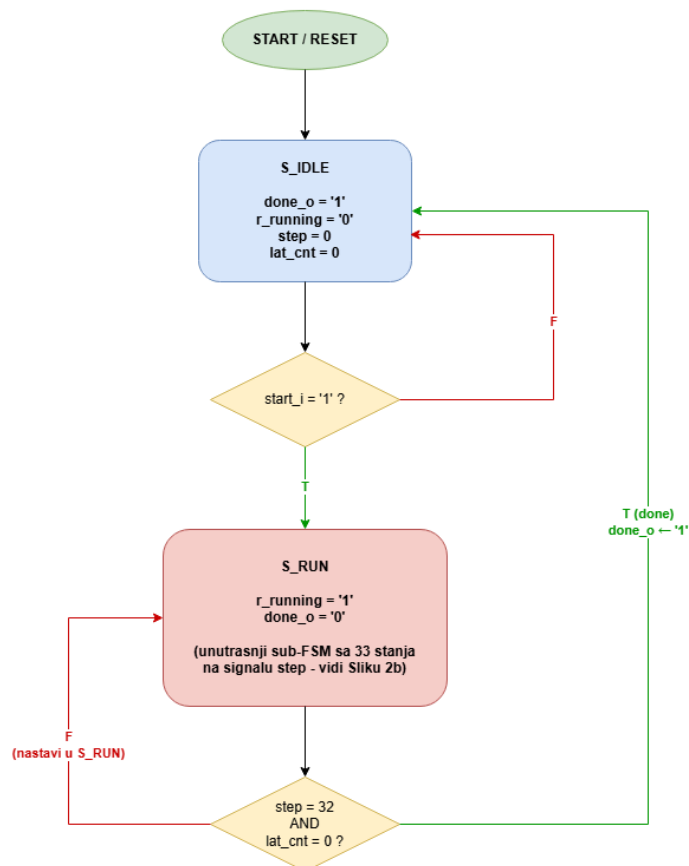
ukupno 86)

- Konverzija float -> Q2.24 fixed-point

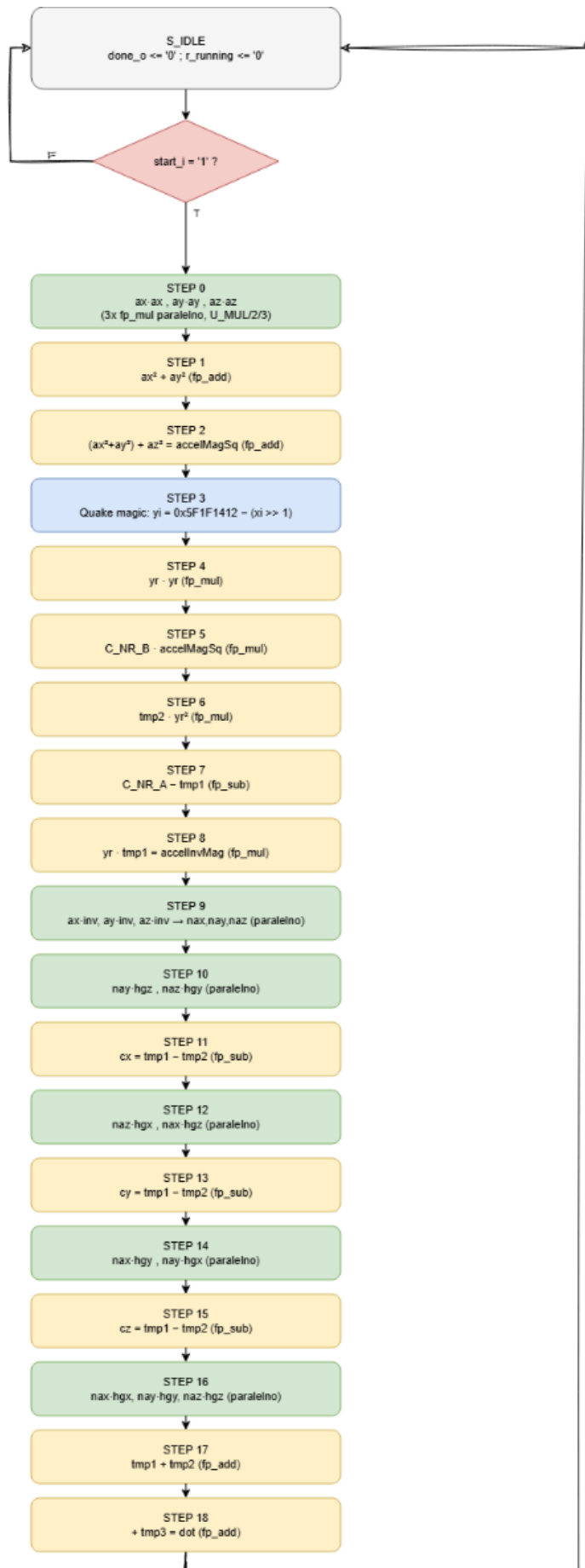
-Postavljanje *done_o* <= '1' i povratak u *S_IDLE*

ASM dijagram je prikazan na Slici 2.

ASM dijagram - ahrs_sec5 (Spoljasnji FSM, 2 stanja)



Slika 2a. ASM dijagram sekcije 5



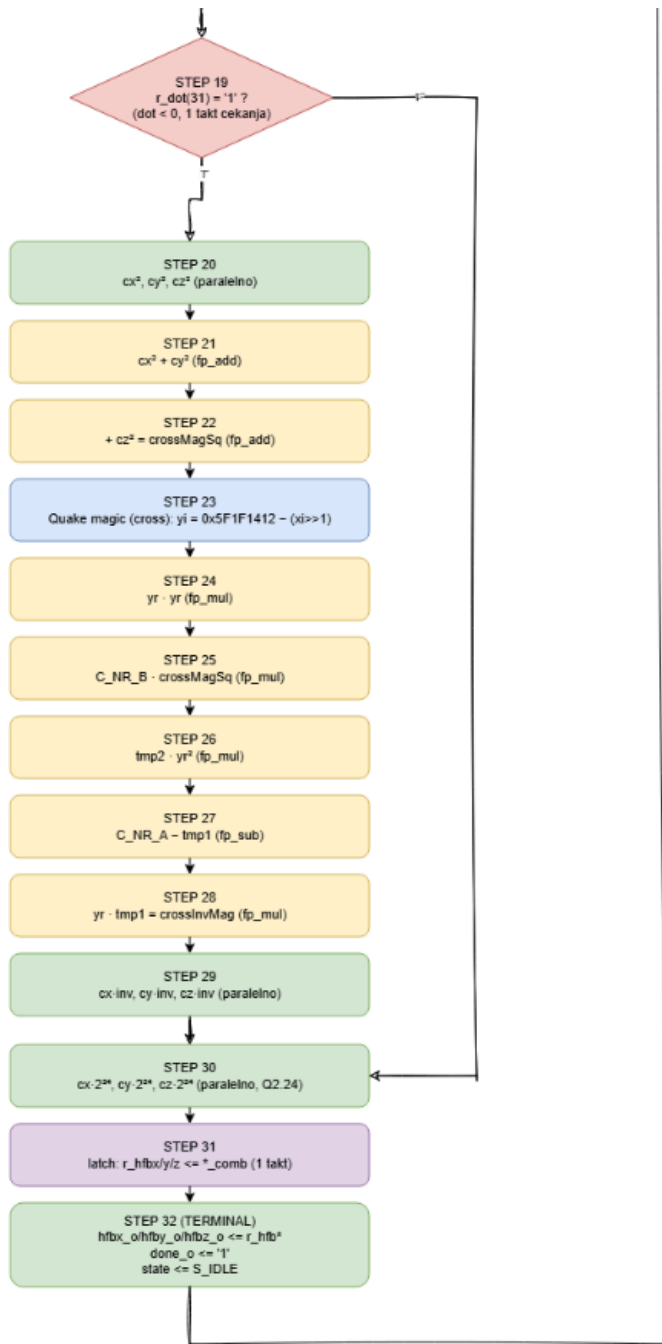
NAPOMENA:

Signal 'step' je registar unutar stanja S_RUN spoljasnijeg FSM-a. step se NE menja svaki takt automatski - svaki 'FP korak' (zeleno/zuto) cekaa lat_cnt da izbroji C_FP_LAT (4 takta) preko tvalid/tready pre prelaska na sledeci step.

Koraci sa Quake bit-trikom i latch operacijama (plavo/ljubicasto: 3, 19, 23, 31) cekaju samo JEDAN dodatni takt.

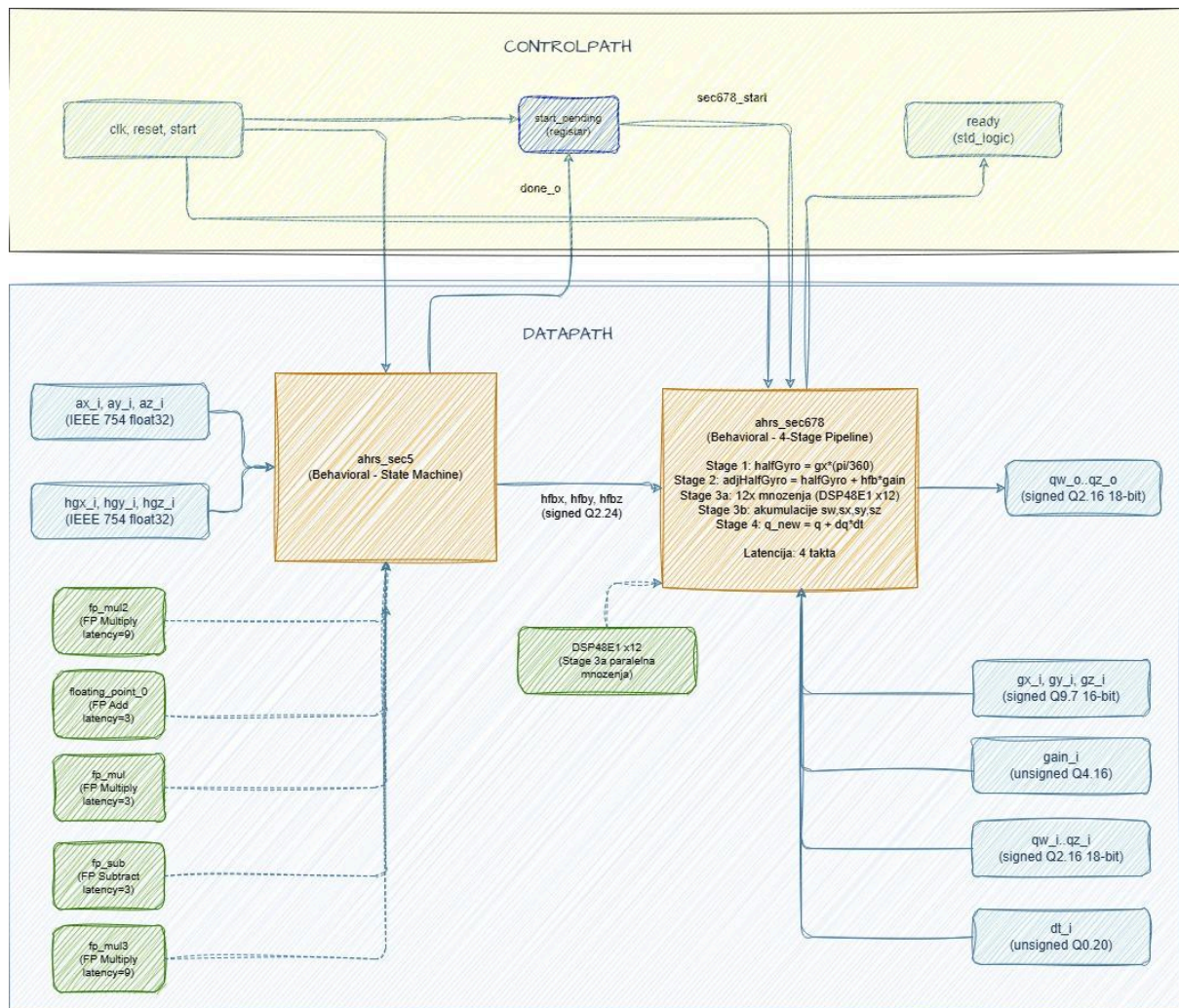
33 koraka = 33 stanja FSM-a, ali svako stanje traje VISE od jednog takta.

Latencija sekcije 5:
58 taktova ako dot >= 0 (F grana)
86 taktova ako dot < 0 (T grana, puna normalizacija cross vektora, koraci 20-29)



Slika 2b. ASM dijagram sekcije 5 - sa detaljnijim S_RUN stanjem

5. Blok dijagram komponenti



Slika 3. Blok dijagram sistema

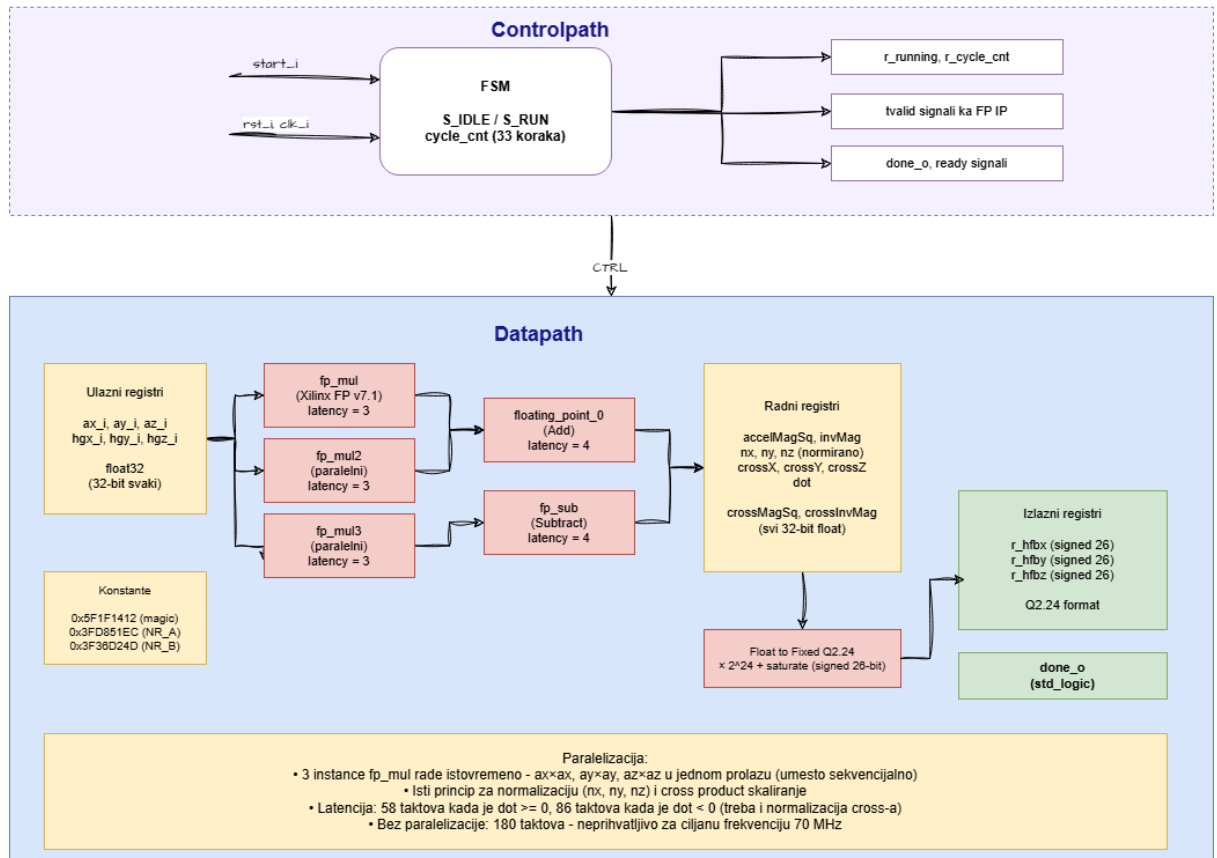
Dizajn se sastoji od dva glavna bloka instancirana unutar strukturnog top-level entiteta `ahrs_sec5to8`. Blok `ahrs_sec5` implementira kompenzaciju akcelerometra kao state machine i koristi tri tipa Xilinx Floating Point v7.1 IP jezgara, ukupno pet instanci: množać `fp_mul` (instanciran tri puta paralelno kao `U_MUL`, `U_MUL2` i `U_MUL3`, sve sa identičnom latencijom od 4 takta), sabirač `floating_point_0` (instanca `U_ADD`) i oduzimač `fp_sub` (instanca `U_SUB`). Blok `ahrs_sec678` implementira sekcije 6, 7 i 8 kao pipeline.

Control path obuhvata signale `clk`, `reset` i `start`, zatim `start_pending` registar koji sinhronizuje pokretanje dva bloka, `done_o` signal kojim sekcija 5 signalizira završetak, `sec678_start` koji okida pipeline sekcije 678, te `ready` signal na izlazu koji označava da je novi kvaternion dostupan.

Datapath počinje ulaznim float32 vrednostima akcelerometra i vektora gravitacije koje ulaze u sekciju 5. Rezultat sekcije 5 su vrednosti `hfbx`, `hfby` i `hfbz` u formatu Q2.24, koje zajedno sa vrednostima žiroskopa, gain faktorom, trenutnim kvaternionom i vremenskim korakom ulaze u sekciju 678. Na izlazu se dobija ažurirani kvaternion u formatu Q2.16.

5.1 Detaljna struktura sekcije 5

ahrs_sec5 - Datapath i Controlpath

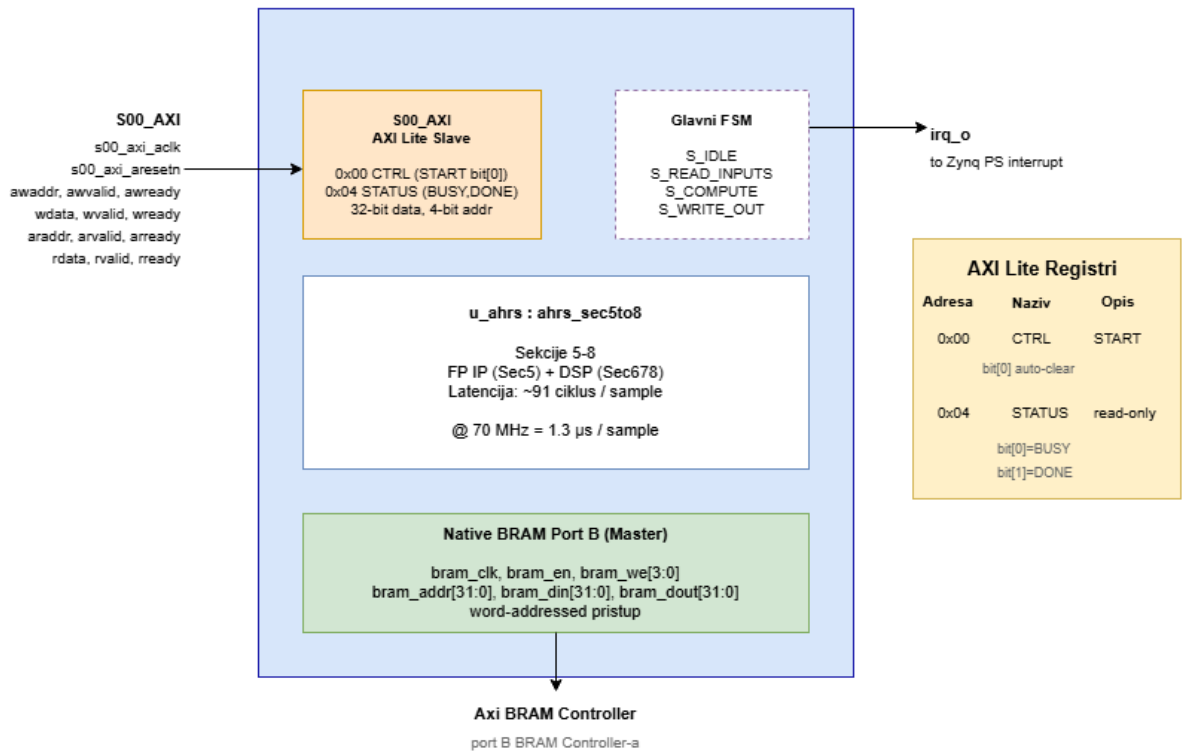


Slika 4. Struktura sekcije 5

Sekcija 5 sadrži FSM (state machine) sa dva stanja i ciklus brojač koji indeksira korake u S_RUN stanju. Datapath obuhvata ulazne registre za sve floating-point vrednosti, tri paralelne instance fp_mul IP jezgra (U_MUL, U_MUL2, U_MUL3), jednu instancu floating_point_0 (U_ADD, sabirač) i jednu instancu fp_sub (U_SUB, oduzimač), kao i tabelu sa konstantama (0x5F1F1412 za magic number, koeficijenti Newton-Raphson korekcije). Sve tri paralelne instance množača koriste isti IP (fp_mul) sa identičnom konfiguracijom. Nakon sinteze Vivado ih u hijerarhiji prikazuje kao fp_mul, fp_mul_1 i fp_mul_2 (auto-suffix imenovanje replika istog izvornog jezgra).

6. Pakovanje IP jezgra

ahrs_ip_v3_0 - IP jezgro (nakon pakovanja)



Slika 5. IP jezgro nakon pakovanja

Da bi se omogućila komunikacija sa procesorom, ahrs_sec5to8 je upakovan u ahrs_ip_v3_0 IP jezgro sa AXI Lite Slave portom za kontrolu i native BRAM Port B master interfejsom za pristup BRAM memoriji.

AXI Lite Registri:

Adresa	Naziv	Tip	Opis
0x00	CTRL	Write	bit[0] = START (auto-clear nakon jednog ciklusa)
0x04	STATUS	Read	bit[0] = BUSY, bit[1] = DONE

Memory Map u BRAM-u (word offsets):

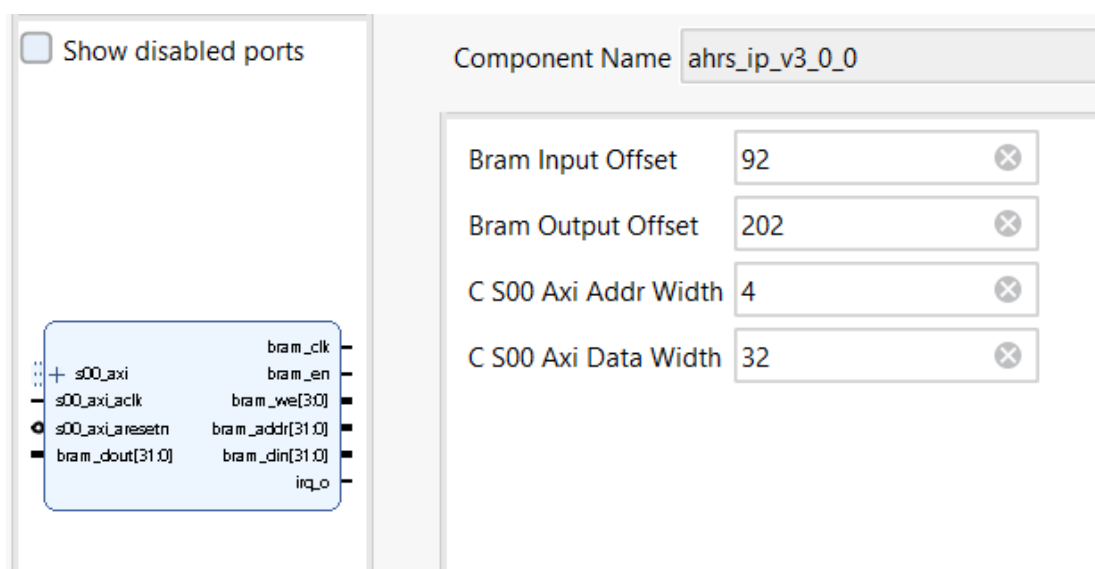
Word offset	Sadržaj
92 (0x5C)	INPUT_BUFFER: gx, gy, gz, ax, ay, az, hgx, hgy, hgz, dt, gain, qw, qx, qy, qz (15 reči)
202 (0xCA)	OUTPUT_BUFFER: qw_out, qx_out, qy_out, qz_out (4 reči)

Konverzija float - fixed (u IP-ju):

Signal	Format	Konverzija
gx, gy, gz	signed Q9.7 (16-bit)	donjih 16 bita float reprezentacije
gain	unsigned Q4.16 (20-bit)	donjih 20 bita
dt	unsigned Q0.20 (20-bit)	donjih 20 bita
qw, qx, qy, qz	signed Q2.16 (18-bit)	donjih 18 bita
ax, ay, az, hgx, hgy, hgz	IEEE 754 float32	direktno kao std_logic_vector(31:0)

Ova konverzija se obavlja u IP-ju jer su podaci u BRAM-u u float formatu (zbog uniformnog memorijskog interfejsa sa CPU-om).

IP takođe ima i irq_o izlaz koji se aktivira kada je obrada gotova, i povezuje se sa IRQ_F2P portom Zynq PS-a kako bi mogao da generiše interrupt.



Slika 6. Izgled IP jezgra u Vivado IP Catalog-u

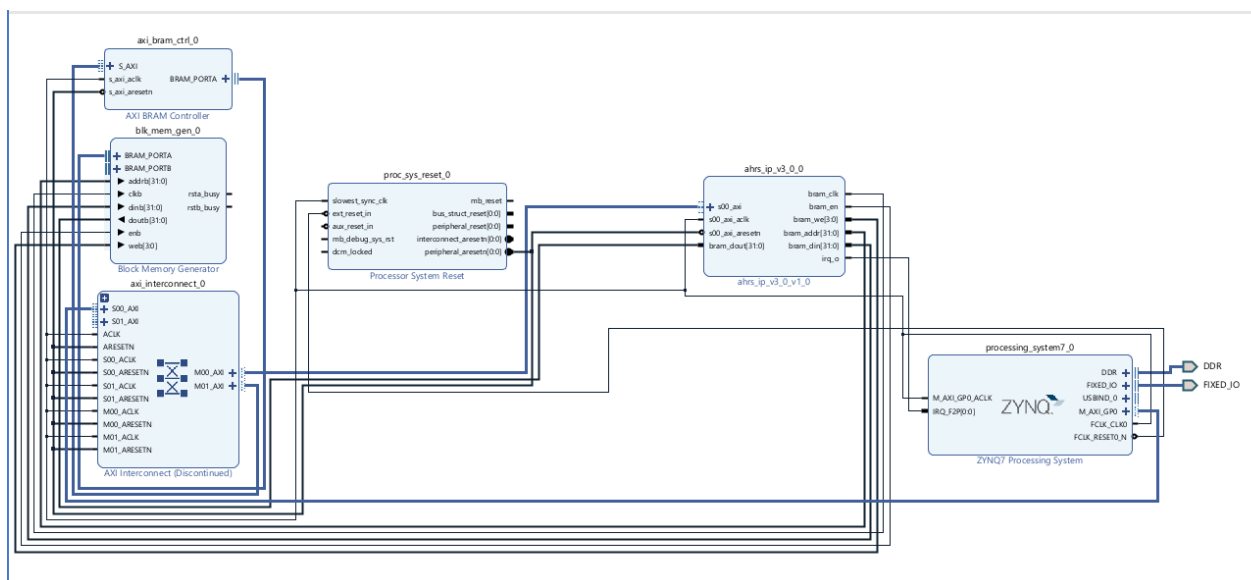
7. Analiza integrisanog sistema nakon sinteze i implementacije

7.1 Izgled integrisanog sistema

Integrirani sistem se sastoji od sledećih komponenti:

- Zynq 7 Processing System (ARM Cortex-A9 procesor)
- ahrs_ip_v3_0 (naš AHRS IP)
- AXI BRAM Controller - kontroler za pristup BRAM memoriji putem AXI interfejsa
- Block Memory Generator - fizička 2 KB BRAM memorija (512 reči × 32-bit)
- AXI Interconnect - povezuje M_AXI_GP0 procesora sa S00_AXI portovima IP-ja i BRAM kontrolera
- Processor System Reset - distribucija reset signala

Zynq PS je konfigurisan tako što su omogućene periferije koje koristimo (UART za debug ispis), povezan je DDR interfejs, postavljen je jedan pin za IRQ (kako bi IP mogao da signalizira završetak), i podešen je M_AXI_GP0 interfejs. Taktni signal od 70 MHz dobija se direktno iz FCLK_CLK0 izlaza PS7 bloka, bez dodatnog Clocking Wizard jezgra. Clocking Wizard je bio prisutan u ranijoj fazi razvoja, dok je dizajn testiran kao standalone top_zybo entitet pre pakovanja u IP, i korišćen je za podešavanje radne frekvencije tokom verifikacije tajminga. U finalnoj IP verziji ova funkcija preuzeta je od PS7 koji direktno generiše 70 MHz.



Slika 7. Blok šema sistema u Vivado IP Integrator-u (screenshot iz alata)

7.2 Utrošeni resursi

Name	Slice LUTs (17600)	Slice Registers (35200)	Slice (4400)	LUT as Logic (17600)	LUT as Memory (6000)	Block RAM Tile (60)	DSPs (80)
design_1_wrapper	3208	3731	1409	3168	40	2	55
design_1_i (design_1)	3208	3731	1409	3168	40	2	55
ahrs_ip_v3_0_0 (design_1_ahrs_ip_v3_0_0_1)	2082	2418	956	2082	0	0	55
U0 (ahrs_ip_v3_0)	2082	2418	956	2082	0	0	55
u_ahrs (ahrs_sec5to8)	1941	1997	814	1941	0	0	55
sec5 (ahrs_sec5)	1849	1590	667	1849	0	0	10
U_ADD (floating_point_0)	172	109	52	172	0	0	2
U_MUL (fp_mul_1)	73	90	34	73	0	0	2
U_MUL2 (fp_mul_2)	73	90	35	73	0	0	2
U_MUL3 (fp_mul)	73	90	37	73	0	0	2
U_SUB (fp_sub)	173	109	67	173	0	0	2
sec678 (ahrs_sec678)	90	406	150	90	0	0	45
axi_bram_ctrl_0 (design_1_axi_bram_ctrl_0_0)	259	252	99	251	8	0	0
axi_interconnect_0 (design_1_axi_interconnect_0)	848	1018	374	819	29	0	0

Slika 8. Utrošeni resursi nakon implementacije integrisanog sistema

Iz tabele se vidi da integrisani sistem koristi 3208 LUT-ova (18.2% od ukupno 17 600 dostupnih), 3731 flip-flop-a (10.6% od 35 200), 55 DSP jedinica (68.7% od 80), 1409 slice-ova (32%) i 2 BRAM bloka. Najveći deo resursa zauzima sam AHRS IP (ahrs_ip_v3_0) sa 2082 LUT-a i 55 DSP-ova.

Unutar AHRS IP-ja, sekcija 5 (ahrs_sec5) koristi 10 DSP raspoređenih u pet instanci Xilinx Floating Point IP jezgara: sabirač U_ADD (floating_point_0) koristi 2 DSP, tri paralelne instance množača U_MUL, U_MUL2 i U_MUL3 (sve nastale od istog izvornog jezgra fp_mul, instancirane u VHDL kodu kao U_MUL, U_MUL2 i U_MUL3) po 2 DSP, i oduzimač U_SUB (fp_sub) još 2 DSP. Sekcija 678 koristi preostala 45 DSP za pipeline množenja raspoređena po svim stage-ovima: 3 DSP u Stage 1 za konverziju žiroskopa (gx , gy , $gz \times \pi/360$), 3 DSP u Stage 2 za korekciju gain faktorom ($hfb \times gain$), 12 DSP u Stage 3a za paralelna množenja kvaterniona sa korigovanom ugaonom brzinom, i 4 DSP u Stage 4 za Ojlerovu integraciju ($dq \times dt$). Ostatak do ukupnih 45 DSP Vivado alocira zbog širine operanada pojedinih množenja.

AXI Interconnect dodaje 848 LUT-ova i 1018 flip-flop-ova, što je tipično za AXI infrastrukturu koja vrši rutiranje između master-a i više slave-ova. AXI BRAM Controller koristi 259 LUT-ova i 252 flip-flop-a. Ukupno utrošeni resursi pokazuju da sistem zauzima manje od trećine LUT-ova i flip-flop-ova na xc7z010 čipu, ostavljajući dosta prostora za buduća proširenja algoritma ili dodatne IP-ove. DSP utilization od 68.7% je relativno visok, ali je očekivan s obzirom na to da je AHRS algoritam aritmetički intenzivan.

7.3 Kritična putanja i maksimalna frekvencija

Name	Waveform	Period (ns)	Frequency (MHz)
clk_fpga_0	{0.000 7.000}	14.000	71.429

Slika 9. Maksimalna frekvencija integrisanog sistema

Slika 9 prikazuje Clock Summary izveštaj iz Vivado-a. Sistem radi na frekvenciji od 71.429 MHz (period 14 ns), što je neznatno iznad ciljane frekvencije od 70 MHz koja je postavljena u S7 konfiguraciji (FCLK_CLK0). Ova mala razlika je posledica činjenice da Vivado pri implementaciji ostavlja malu rezervu (slack) kako bi izbegao negativan timing budget.

Summary						
Name	↳ Path 1					
Slack	2.056ns					
Source	design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_tmp3_reg[25]/C (rising edge-triggered cell FDRE clocked by clk_fpga_0 [rise@0.000ns fall@7.000ns period=14.000ns])					
Destination	design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz_reg[9]/D (rising edge-triggered cell FDRE clocked by clk_fpga_0 [rise@0.000ns fall@7.000ns period=14.000ns])					
Path Group	clk_fpga_0					
Path Type	Setup (Max at Slow Process Corner)					
Requirement	14.000ns (clk_fpga_0 rise@14.000ns - clk_fpga_0 rise@0.000ns)					
Data Path Delay	11.283ns (logic 4.576ns (40.557%) route 6.707ns (59.443%))					
Logic Levels	17 (CARRY4=9 LUT4=5 LUT5=2 LUT6=1)					
Clock Path Skew	-0.145ns					
Clock U..tainty	0.213ns					
Source Clock Path						
Delay Type	Incr (ns)	Path ...	Location	Cell Pin	Cell	Netlist Resources
(clock clk_fpga_0 rise edge)	(n) 0.000	0.000				
PS7	(n) 0.000	0.000	Site: PS7_X0Y0	FCLKCLK[0]	PS7_i (PS7)	design_1_i/processing_system7_0/inst/PS7_i/FCLKCLK[0]
net (fo=1, routed)	1.207	1.207				design_1_i/processing_system7_0/inst/FCLK_CLK_unbuffered[0]
BUFG (Prop. bufg 1.0)	(n) 0.101	1.308	Site: BUF...TRL_X0Y15	O	buffer_fclk_clk_0.FCLK_CLK_0_BUFG (BUFG)	design_1_i/processing_system7_0/inst/buffer_fclk_clk_0.FCLK_CLK_0_BUFG/O
net (fo=3827, routed)	1.649	2.957				design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/clk_i
FDRE			Site: SLICE_X24Y25	C	r_tmp3_reg[25] (FDRE)	design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_tmp3_reg[25]/C

Slika 10. Kritična putanja integrisanog sistema - Summary

Slika 10 prikazuje sažetak kritične putanje. Slack iznosi 2.056 ns što znači da signal stiže do svoje destinacije sa pozitivnom rezervom od približno 2 ns pre kraja takt-ciklusa. Drugim rečima, sistem bi mogao da radi i na nešto višoj frekvenciji nego što je trenutno postavljeno (procenjeno ~83 MHz), ali ostavljamo rezervu zbog varijabilnosti procesa i temperature. Ukupno data path delay iznosi 11.283 ns, od čega logičko kašnjenje čini 4.576 ns (40.5%) a delay interkonekcije 6.707 ns (59.4%), što je tipično za 7-Series FPGA gde routing često dominira nad logičkim kašnjenjem.

Data Path						
Delay Type	Incr (ns)	Path (...)	Location	Cell Pin	Cell	Netlist Resources
FDRE (Prop fdre C Q)	(r) 0.518	3.475	Site: SLICE_X24Y25	Q	r_tmp3_reg[25] (FDRE)	design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_tmp3_reg[25]/Q
net (fo=11, routed)	0.884	4.359				design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_tmp3_0[25]
LUT4 (Prop lut4 I0 O)	(f) 0.124	4.483	Site: SLICE_X25Y25	O	r_hfbz[25]_i_14 (LUT4)	design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz[25]_i_14/O
net (fo=17, routed)	0.597	5.080				design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz[25]_i_14_n_0
LUT4 (Prop lut4 I2 O)	(r) 0.124	5.204	Site: SLICE_X25Y26	O	r_hfbz[25]_i_7 (LUT4)	design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz[25]_i_7/O
net (fo=91, routed)	0.670	5.874				design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz[22]_i_15_n_0
LUT6 (Prop lut6 I5 O)	(r) 0.124	5.998	Site: SLICE_X25Y30	O	r_hfbz[23]_i_26 (LUT6)	design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz[23]_i_26/O
net (fo=1, routed)	0.000	5.998				design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz[23]_i_26_n_0
CARRY4 (Prop c_4 S[11] CO[3])	(r) 0.550	6.548	Site: SLICE_X25Y30	CO[3]	r_hfbz_reg[23]_i_10 (CARRY4)	design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz_reg[23]_i_10/CO[3]
net (fo=1, routed)	0.000	6.548				design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz_reg[23]_i_10_n_0
CARRY4 (Prop carry4 CI CO[3])	(r) 0.114	6.662	Site: SLICE_X25Y31	CO[3]	r_hfbz_reg[23]_i_28 (CARRY4)	design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz_reg[23]_i_28/CO[3]
net (fo=1, routed)	0.000	6.662				design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz_reg[23]_i_28_n_0
CARRY4 (Prop carry4 CI CO[3])	(r) 0.114	6.776	Site: SLICE_X25Y32	CO[3]	r_hfbz_reg[23]_i_18 (CARRY4)	design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz_reg[23]_i_18/CO[3]
net (fo=1, routed)	0.000	6.776				design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz_reg[23]_i_18_n_0
CARRY4 (Prop carry4 CI CO[3])	(r) 0.114	6.890	Site: SLICE_X25Y33	CO[3]	r_hfbz_reg[23]_i_20 (CARRY4)	design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz_reg[23]_i_20/CO[3]
net (fo=1, routed)	0.000	6.890				design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz_reg[23]_i_20_n_0
CARRY4 (Prop carry4 CI CO[3])	(r) 0.114	7.004	Site: SLICE_X25Y34	CO[3]	r_hfbz_reg[23]_i_19 (CARRY4)	design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz_reg[23]_i_19/CO[3]
net (fo=1, routed)	0.000	7.004				design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz_reg[23]_i_19_n_0
CARRY4 (Prop carry4 CI O[1])	(r) 0.334	7.338	Site: SLICE_X25Y35	O[1]	r_hfbz_reg[23]_i_17 (CARRY4)	design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz_reg[23]_i_17/O[1]
net (fo=2, routed)	0.891	8.229				design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz_reg[23]_i_17_n_6
LUT4 (Prop lut4 I3 O)	(r) 0.303	8.532	Site: SLICE_X26Y31	O	r_hfbz[23]_i_21 (LUT4)	design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz[23]_i_21/O
net (fo=2, routed)	0.818	9.350				design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz[23]_i_21_n_0
LUT5 (Prop lut5 I4 O)	(r) 0.152	9.502	Site: SLICE_X27Y32	O	r_hfbz[23]_i_8 (LUT5)	design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz[23]_i_8/O
net (fo=1, routed)	0.817	10.320				design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz[23]_i_8_n_0
LUT4 (Prop lut4 I1 O)	(r) 0.352	10.672	Site: SLICE_X24Y32	O	r_hfbz[23]_i_2 (LUT4)	design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz[23]_i_2/O
net (fo=29, routed)	0.668	11.339				design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz[23]_i_2_n_0
LUT5 (Prop lut5 I1 O)	(r) 0.328	11.667	Site: SLICE_X22Y29	O	r_hfbz[4]_i_12 (LUT5)	design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz[4]_i_12/O
net (fo=1, routed)	0.000	11.667				design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz[4]_i_12_n_0
CARRY4 (Prop c_4 S[11] CO[3])	(r) 0.550	12.217	Site: SLICE_X22Y29	CO[3]	r_hfbz_reg[4]_i_8 (CARRY4)	design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz_reg[4]_i_8/CO[3]
net (fo=1, routed)	0.000	12.217				design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz_reg[4]_i_8_n_0
CARRY4 (Prop carry4 CI CO[3])	(r) 0.114	12.331	Site: SLICE_X22Y30	CO[3]	r_hfbz_reg[12]_i_7 (CARRY4)	design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz_reg[12]_i_7/CO[3]
net (fo=1, routed)	0.000	12.331				design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz_reg[12]_i_7_n_0
CARRY4 (Prop carry4 CI O[0])	(r) 0.222	12.553	Site: SLICE_X22Y31	O[0]	r_hfbz_reg[12]_i_3 (CARRY4)	design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz_reg[12]_i_3/O[0]
net (fo=1, routed)	0.712	13.266				design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/res30[9]
LUT4 (Prop lut4 I2 O)	(r) 0.325	13.591	Site: SLICE_X21Y31	O	r_hfbz[9]_i_1 (LUT4)	design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz[9]_i_1/O
net (fo=1, routed)	0.649	14.240				design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz_comb[9]
FDRE			Site: SLICE_X18Y31	D	r_hfbz_reg[9] (FDRE)	design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz_reg[9]/D
Arrival Time		14.240				
Destination Clock Path						
Delay Type	Incr (ns)	Path (...)	Location	Cell Pin	Cell	Netlist Resources
(clock clk_fpga_0 rise edge)	(r) 14.000	14.000				
PS7	(r) 0.000	14.000	Site: PS7_X0Y0	FCLKCLK[0]	PS7_i (PS7)	design_1_i/processing_system7_0/inst/PS7_i/FCLKCLK[0]
net (fo=1, routed)	1.101	15.101				design_1_i/processing_system7_0/inst/FCLK_CLK_unbuffered[0]
BUFPG (Prop bufpg I O)	(r) 0.091	15.192	Site: BUF_TRL_X0Y15	O	buffer_fclk_clk_0.FCLK_CLK_0_BUFPG (BUFPG)	design_1_i/processing_system7_0/inst/buffer_fclk_clk_0.FCLK_CLK_0_BUFPG/O
net (fo=3827, routed)	1.490	16.682				design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/clk_i
FDRE			Site: SLICE_X18Y31	C	r_hfbz_reg[9] (FDRE)	design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz_reg[9]/C
clock pessimism	0.130	16.812				
clock uncertainty	-0.213	16.599				
FDRE (Setup fdre C D)	-0.303	16.296	Site: SLICE_X18Y31		r_hfbz_reg[9] (FDRE)	design_1_i/ahrs_ip_v3_0_0/U0/u_ahrs/sec5/r_hfbz_reg[9]
Required Time		16.296				

Slika 11. Kritična putanja integrisanog sistema - Datapath, Destinations Clock Path

Nakon sinteze i implementacije integrisanog sistema, maksimalna frekvencija rada iznosi **70 MHz** (kao što je i konfigurisano u PS7 bloku). Kritična putanja je tipično u Stage 3a sekcije 678 (12 paralelnih DSP množenja).

7.4 Propusna moć i kašnjenje

Pri 70 MHz, vreme jednog HW poziva (latencija + komunikacija preko AXI Lite):

Latencija sec5to8: 91 ciklus = $1.3 \mu\text{s}$

AXI Lite write/read transakcije (15 ulaza, 4 izlaza preko BRAM-a): ~50 ciklusa

Ukupno po sample-u: ~2 μs

Za pun dataset od 1918 sample-ova: $1918 \times 2 \mu\text{s} = \sim 3.8 \text{ ms}$

Stvarno izmereno vreme u Vitis-u uključuje i SW overhead (priprema podataka, normalizacija kvaterniona u SW Sec9-11, formatiranje CSV ispisa za UART). Ukupno vreme od BEGIN_CSV do END_CSV markera u PuTTY logu je oko ~25 sekundi za 1918 sample-ova (dominirano UART ispisom @ 115200 baud).

8. Poređenje sa rezultatima ESL projekta

Na predmetu Projektovanje elektronskih uređaja na sistemskom nivou (PEUNSN) procena za isti algoritam dobijena je pomoću Vitis HLS sinteze. Tabela poređenja ključnih parametara između HLS i RTL implementacije prikazana je u nastavku.

Parametar	ESL (HLS, Vitis HLS)	RTL (Vivado, ručna implementacija)
Target frekvencija	66.7 MHz (clock 15 ns)	70 MHz (clock ~14.3 ns)
DSP utilization	78 / 80	55 / 80 (68.7%)
LUT utilization	9 757 / 17 600	3208 / 17 600 (18.2%)
Latencija po sample-u	65 taktova	91 takt (worst case)
Vreme po sample-u	~975 ns	~1.3 μs

8.1 Diskusija razlika

DSP utilization je manji u RTL verziji (55 naspram 78). Razlog je što HLS automatski raspoređuje mnogo paralelnih množenja na DSP-ove jer optimizuje za propusnu moć, dok RTL sekcija 5 koristi samo tri paralelna FP množača koji se ponovo koriste kroz cikluse FSM-a (10 DSP ukupno za pet FP instanci). Sekcija 678 koristi 45 DSP raspoređenih po svim pipeline stage-ovima: 3 u Stage 1, 3 u Stage 2, 12 u Stage 3a i 4 u Stage 4, a ostatak Vivado alokira zbog širine operanada.

Latencija po sample-u je veća u RTL verziji (91 naspram 65 taktova u najgorem slučaju). Razlog je što HLS koristi pipeline-ovan pristup i za sekciju 5 kroz loop unrolling i loop pipelining, dok je RTL sec5 implementiran kao state machine sa serijskim koracima zbog dugačke latencije FP IP-ova (3-4 takta po operaciji).

Maksimalna frekvencija je slična (66.7 naspram 70 MHz). Ovo pokazuje da ručna pipeline optimizacija u sec678, konkretno razbijanje Stage 3 na 3a i 3b, postiže slične rezultate kao HLS automatska optimizacija.

Glavna prednost RTL pristupa je značajna ušteda DSP resursa, što ostavlja prostora za dodatnu logiku u istom čipu. Zynq xc7z010 ima samo 80 DSP jedinica, pa bi HLS verzija ostavila svega 2 DSP-a slobodna, dok RTL verzija ostavlja oko 25 DSP-ova slobodnih (80 – 55), naspram samo 2 slobodna u HLS verziji, što je i dalje značajna prednost.

9. Testiranje rada u Vitis-u

Nakon sinteze i implementacije, generisan je bitstream i sistem je spušten na Zybo Z7-10 ploču. U Vitis-u je napisana bare-metal aplikacija koja u realnom vremenu komunicira sa AHRS IP-jem preko AXI Lite interfejsa i BRAM memorije, obrađuje 1092 IMU sample-ova iz test dataset-a i ispisuje rezultate preko UART-a.

9.1 Tok obrade jednog sample-a

Za svaki od 1092 sample-ova, aplikacija prolazi kroz sledeći redosled koraka:

1. **SW Sec1-4:** Računa halfGravity vektor iz trenutnog kvaterniona i primenjuje ramped gain faktor koji se postepeno povećava na početku obrade.
2. **Upis ulaza u BRAM:** Procesor upisuje 15 ulaznih reči (gx, gy, gz, ax, ay, az, hgx, hgy, hgz, dt, gain, qw, qx, qy, qz) u BRAM na word offset 92 (0x5C).
3. **Pokretanje IP-ja:** Upisuje se 1 u CTRL registar AHRS IP-ja na adresi 0x40000000, čime se aktivira bit START koji pokreće hardversku obradu.
4. **Polling:** Procesor polling-uje STATUS registar (0x40000004) sve dok ne vidi postavljen DONE bit, što označava završetak obrade.
5. **Čitanje izlaza:** Procesor čita 4 izlazne reči iz BRAM-a sa offset-a 202 (0xCA): qw, qx, qy i qz nove orijentacije.
6. **SW Sec9-11:** Normalizuje kvaternion množenjem sa $\text{fast_inv_sqrt}(qw^2+qx^2+qy^2+qz^2)$, zatim računa Euler uglove (roll, pitch, yaw) po ZYX konvenciji i konvertuje u stepene.
7. **UART ispis:** Ispisuje jedan CSV red preko UART-a sa svim relevantnim podacima trenutnog sample-a.

9.2 Format CSV ispisa

UART output je struktuiran kao CSV sa header-om, što omogućava direktno učitavanje u Python skriptu za vizualizaciju. Format jednog reda je sledeći:

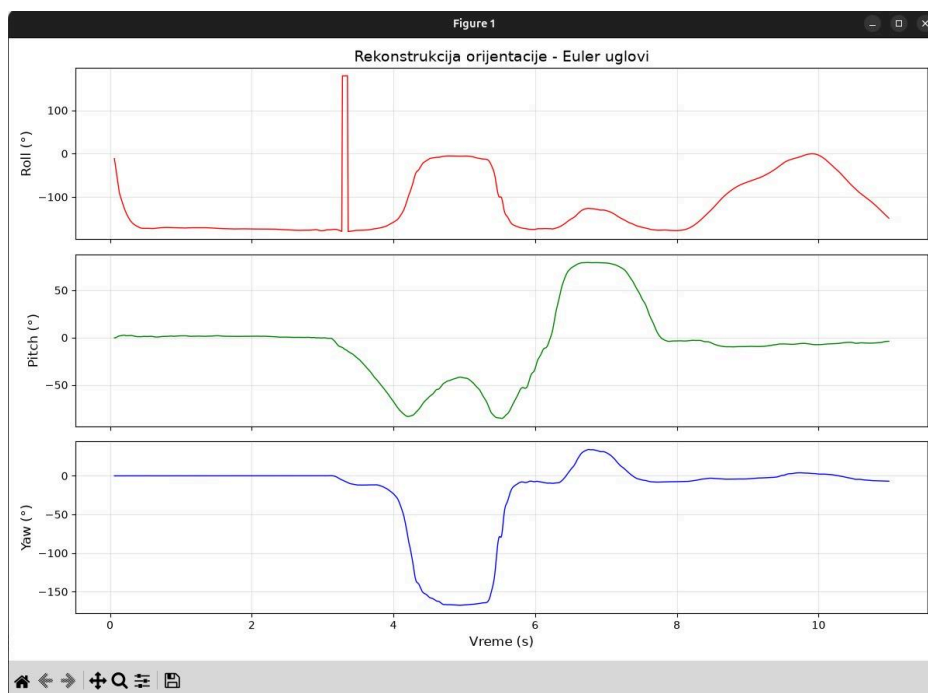
Kolona	Izvor	Opis
time	SW	Vremenski trenutak sample-a (sekunde)
qw, qx, qy, qz	HW + SW norm.	Kvaternion nakon HW obrade i softverske normalizacije
roll, pitch, yaw	SW	Euler uglovi izračunati iz kvaterniona (Sec 9-11)
hfbx, hfby, hfbz	HW (Sec5)	Debug izlaz iz sekcije 5 (akcelerometar feedback)
ax, ay, az	Senzor	Sirovi podaci akcelerometra (za referencu)
hgx, hgy, hgz	SW (Sec1-4)	Half-gravity vektor (za referencu)

Ispis počinje markerom BEGIN_CSV i završava se markerom END_CSV, što olakšava parsiranje u Python skripti.

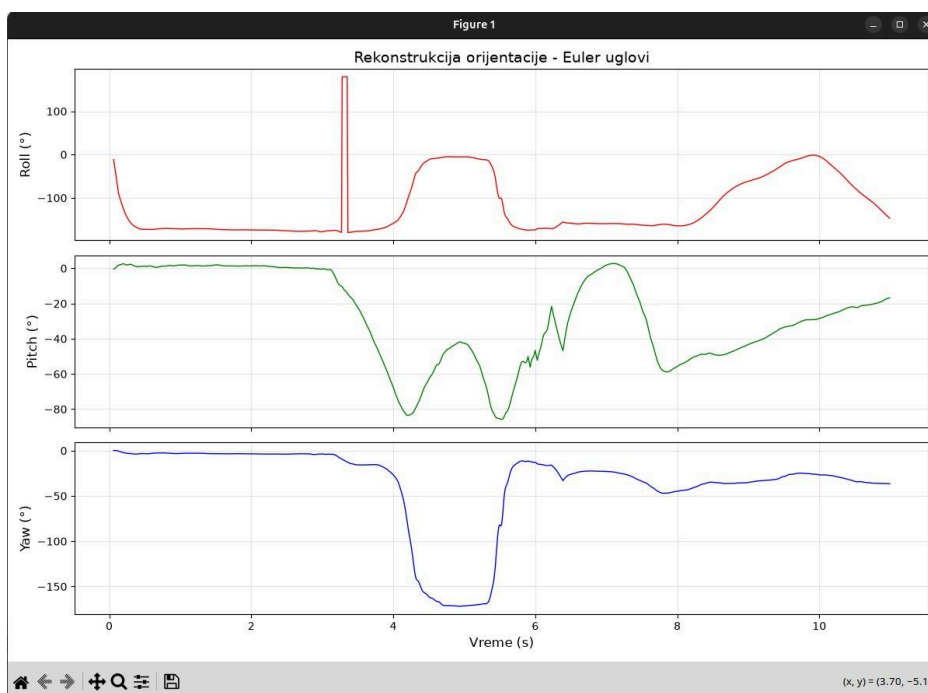
9.3 Verifikacija ispravnosti

Ispravnost hardverske implementacije proverena je poređenjem sa softverskim referentnim modelom u floating-point aritmetici, koji je razvijen u sklopu ESL projekta. Python skripta učitava CSV izlaz, rekonstruiše orijentaciju iz kvaterniona i prikazuje Euler uglove (roll, pitch i yaw) kao funkciju vremena.

Na slikama u nastavku prikazane su rekonstrukcije orijentacije iz HW izlaza i SW reference za isti dataset od 1092 sample-a. Vizuelno poređenje pokazuje da hardverska implementacija prati softversku referencu sa odličnom tačnošću kroz ceo dataset, sve karakteristične promene u orijentaciji (rotacije, povratak u početnu poziciju) jasno se vide u oba grafika.



Slika 12. Rekonstrukcija orijentacije iz SW reference (Roll, Pitch, Yaw)



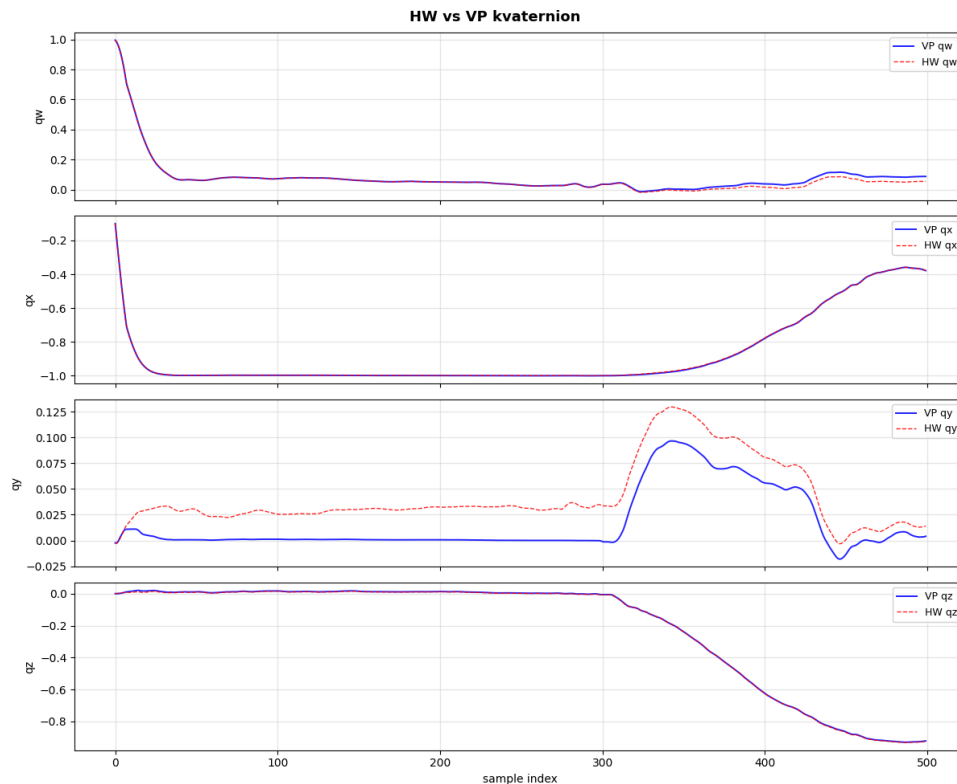
Slika 13. Rekonstrukcija orijentacije iz HW reference (Roll, Pitch, Yaw)

Na graficima možemo uočiti da postoje mala odstupanja između SW i HW podataka, ali vizuelno, korišćenjem 3D prikaza, čovek ne može, golim okom, primetiti razliku kada gleda prikaz okretanja objekta (u našem slučaju, podaci su snimljeni okretanjem mobilnog telefona).

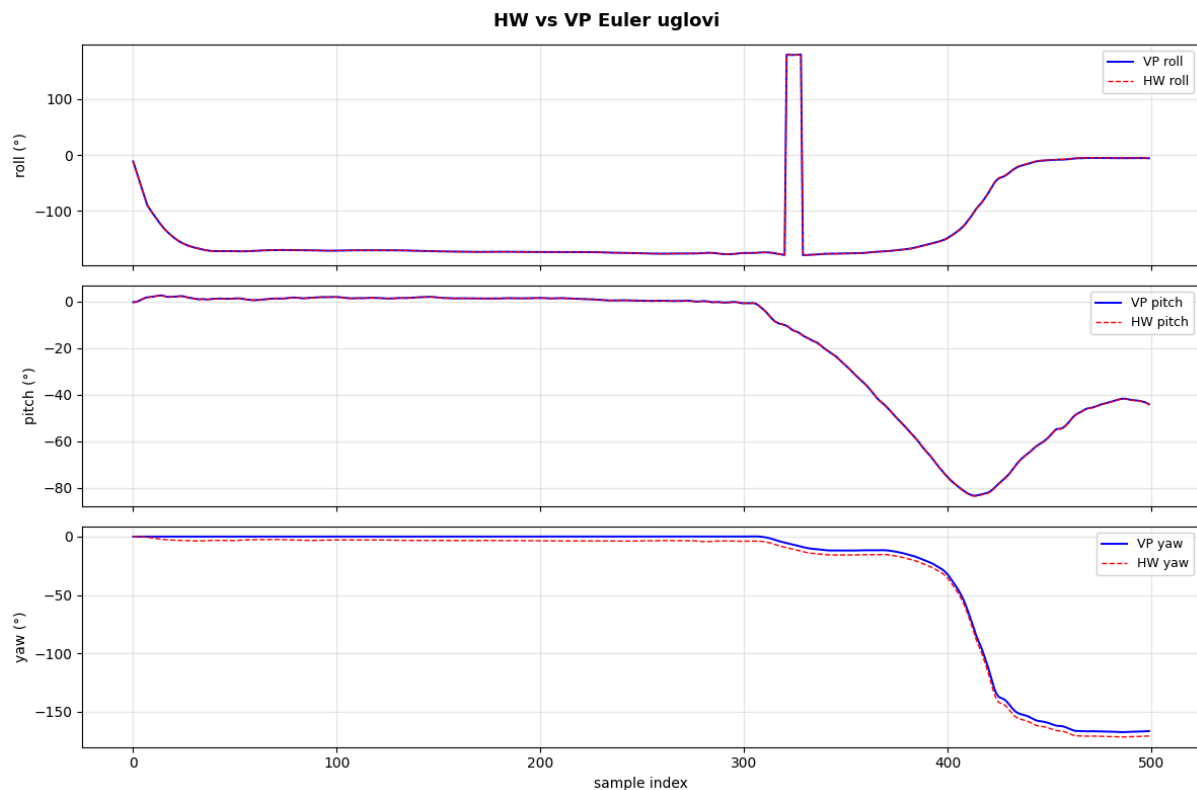
9.4 Poređenje sa hardverskim modelom iz virtuelne platforme

Poređenje iz sekcije 9.3 upoređuje hardverski izlaz sa softverskom float referencom. Da bismo verifikovali RTL implementaciju u odnosu na sistemsku specifikaciju definisanu u ESL projektu, uradili smo dodatno poređenje sa hardverskim modelom iz virtuelne platforme (VP). VP koristi SystemC `sc_fixed` sa istim Q formatima za sekcije 6-8 kao RTL (Q9.7 za žiroskop, Q2.20 za halfGyro, Q5.21 za adjHalfGyro, Q2.16 za kvaternion).

Slika 14 prikazuje prekrivanje kvaterniona za oba modela, a Slika 15 prikazuje Euler uglove (roll, pitch, yaw) izračunate iz tih kvaterniona.



Slika 14. Prekrivanje HW vs VP kvaterniona



Slika 15. HW vs VP Euler uglovi

Rezultati poređenja pokazuju da HW i VP predstavljaju istu fizičku rotaciju:

- Norma kvaterniona je identična kod oba modela (mean = 1.000124, std $\approx 1.39 \cdot 10^{-5}$), što potvrđuje da obe implementacije održavaju kvaternion kao validnu rotaciju.
- Euler uglovi se praktično poklapaju - roll, pitch i yaw krive u oba modela su vizuelno nerazpoznatljive (Slika 15), što znači da je fizička orijentacija koju izračunavaju identična.
- Komponente kvaterniona qx i qz se poklapaju sa razlikom manjom od 100 LSB Q2.16 u proseku ($\sim 0.15\%$ od jedinice).

Male numeričke razlike koje postoje između HW i VP kvaterniona posledica su različite implementacije sekcije 5:

- **VP** implementira sekciju 5 kao hibrid: `sc_fixed` međukoraci u različitim Q formatima (`accel_t` = Q3.17, `magsq_t` = Q2.20, `norm_t` = Q2.24), sa Quake `fast_inv_sqrt` korakom koji privremeno prelazi u IEEE 754 float pa se vraća u fiksni-point kao `invmag_t` = Q6.18.
- **RTL** implementira sekciju 5 potpuno u IEEE 754 float32 kroz Xilinx Floating Point v7.1 IP jezgra (5 instanci: $3 \times \text{fp_mul}$, $1 \times \text{fp_add}$, $1 \times \text{fp_sub}$), sa konverzijom u Q2.24 tek na finalnom izlazu.

Iako se izlazni format sekcije 5 (Q2.24 signed 26-bit) poklapa u oba modela, put do tog izlaza je numerički drugačiji, što uvodi male razlike u pojedinačnim komponentama kvaterniona. Ove razlike se ne akumuliraju u geometrijskom smislu - norma ostaje očuvana, a Euler uglovi se poklapaju, što potvrđuje da RTL implementacija verno prati sistemsku specifikaciju.

9.5 Tačnost i akumulacija greške

Sample-by-sample numeričko poređenje q_w , q_x , q_y i q_z vrednosti pokazuje sledeće ponašanje:

- **Za prvih oko 600 sample-ova:** razlika između HW i SW reference je ispod $1e-3$ po komponenti kvaterniona, što odgovara slaganju na oko 3 decimale.
- **Nakon 600 sample-ova:** razlika počinje postepeno da raste zbog akumulacije fixed-point greške u Q2.16 reprezentaciji kvaterniona.

Ovo ponašanje je očekivano. Q2.16 format ima samo 16 bita razlomka, što daje rezoluciju od približno $1/65536$ ili 1.5×10^{-5} po komponenti kvaterniona. Kroz 1092 uzastopnih iteracija kvaternionске integracije, ova kvantizaciona greška se akumulira i konačno počinje da utiče na orijentaciju. Uprkos tome, kao što se vidi na slikama 12 i 13, glavna struktura orijentacije i sve značajne promene su očuvane, HW implementacija je upotrebljiva za realne primene IMU senzorne fuzije.

Norme kvaterniona dodatno potvrđuju ispravnost: na nekoliko reprezentativnih sample-ova (1, 200, 600, 1000, 1092) izmerene norme su sve unutar 0.05% od jedinice, što znači da kvaternioni predstavljaju validne rotacije bez drift-a u dužini vektora.

9.6 ILA kao dodatna provera

Za dodatno debugovanje i posmatranje internih signala, koristi se Vivado Hardware Manager sa ILA (Integrated Logic Analyzer) IP-jem. ILA omogućava praćenje AXI Lite handshake signala ($S_AXI_AWVALID$, S_AXI_WDATA , S_AXI_AWADDR) i internih AHRS signala (start, ready, done) u realnom vremenu na ploči, što je korisno za potvrdu da se sva komunikacija odvija po protokolu.

Takođe, PuTTY terminal sa CSV ispisom dostupan je kao alternativni način za posmatranje rezultata u realnom vremenu, svaki sample se ispisuje čim ga procesor obradi, pa se može vizuelno potvrditi da obrada teče bez prekida ili grešaka u protokolu.

10. Problemi tokom razvoja i njihova rešenja

Tokom implementacije RTL verzije AHRS algoritma naišli smo na nekoliko netrivialnih problema koji su zahtevali pažljivu analizu. Većina je proizašla iz neusaglašenosti između onoga što smo očekivali da Vivado uradi i onoga što je on stvarno radio sa našim VHDL kodom, kao i iz suptilnih pitanja vremenskog usklađivanja u state machine-u sekcije 5. U ovoj sekciji opisujemo glavne probleme i kako smo ih rešili, jer mislimo da je to često najkorisniji deo svakog hardverskog projekta, onaj koji se najmanje vidi u finalnoj verziji koda.

10.1 Pogrešna pretpostavka o latenciji FP IP jezgara

Prva velika greška koja nas je zbunila bila je u FSM-u sekcije 5. Pretpostavljali smo da je latencija svakog Xilinx Floating Point v7.1 IP jezgra 3 ciklusa. Ispostavilo se da naša konfiguracija IP jezgra koristi AXI-Stream tvalid/tready handshake sa latencijom od 4 ciklusa od trenutka postavljanja tvalid do kada je rezultat validan na izlazu. Ova razlika od jednog ciklusa značila je da je FSM čitao izlaze IP-ova jedan ciklus pre nego što su zaista bili spremni, pa su naredni koraci radili sa starim vrednostima i konačni rezultati nisu imali nikakve veze sa očekivanim.

Rešenje je bilo prepravljanje cele tabele tajminga u FSM-u tako da svaki korak čeka 4 ciklusa umesto 3, kao i dodavanje brojača ciklusa koji nam je omogućio da u simulaciji merimo ukupnu latenciju i upoređujemo je sa očekivanim vrednostima. Brojač je takođe služio za debugovanje, kada bi se latencija odstupala od očekivane, znali smo da nešto nije u redu sa konkretnim korakom u FSM-u.

10.2 Sekvencijalna konverzija float u fixed kao usko grlo

U prvom pokušaju, konverzija rezultata sekcije 5 iz float32 u Q2.24 signed 26-bit format bila je implementirana kao VHDL funkcija f32_to_s26 koja se pozivala tri puta uzastopno za hfbx, hfby i hfbz. Iako su funkcije u VHDL-u kombinacione, način na koji ih Vivado sintezuje doveo je do toga da je sintezator generisao serijsku evaluaciju, što je dodavalo ciklus po komponenti i kvarilo tajming.

Prešli smo na tri paralelna kombinaciona bloka direktno u arhitekturi, bez funkcije. Sintezator sada vidi tri nezavisna stabla logike i rasporedi ih paralelno, čime je sva konverzija završena u jednom ciklusu. Shvatili smo da iako VHDL funkcije izgledaju dobro za enkapsulaciju logike, kada je vremenski kritično bolje je pisati paralelne grane eksplicitno.

10.3 Vremensko usklađivanje provere dot proizvoda

Sekcija 5 ima tačku grananja gde se proverava da li je skalarni proizvod normalizovanog ubrzanja i vektora gravitacije negativan. Ako jeste, treba dodatno normalizovati vektorski proizvod (što produžava latenciju sa 58 na 86 ciklusa). Inicijalno smo proveravali znak signala dot odmah po dolasku iz `floating_point_0` (sabirača), ali je rezultat na izlazu sabirača bio validan tek u sledećem ciklusu od onog kada smo ga čitali. Posledica je bila pogrešno grananje na otprilike polovini sample-ova: algoritam je sa nekim ulazima išao kroz kraću putanju iako je trebalo da ide kroz dužu, i obrnuto.

Dodavanjem jednog ciklusa kašnjenja pre provere znaka, grananje se uvek dešavalo nakon što je izlaz sabirača stabilizovan. Ovo je još jedan primer istog problema iz tačke 10.1, pogrešna pretpostavka o tome kada je izlaz FP IP-a zaista validan.

10.4 Sinhronizacija nakon Quake inverse sqrt-a

Slično prethodnom problemu, korak fast inverse square root u sekciji 5 koristi nekoliko uzastopnih floating-point operacija, manipulaciju bita za inicijalnu aproksimaciju, množenje, oduzimanje, ponovno množenje sa Newton-Raphson korekcijom. Početna implementacija je čitala registar koji čuva međurezultat pre nego što je dobio konačnu vrednost iz FP IP-a. Morali smo da uvedemo eksplicitno čekajuće stanje u FSM-u između koraka, što je produžilo ukupnu latenciju sekcije 5 ali je obezbedilo deterministički ispravan rezultat.

10.5 Agresivna optimizacija Vivado sintetizatora

Najteži problem za debugovanje pojavio se u sekciji 678 nakon što smo počeli da pokrećemo sistem na ploči. Hardver je davao izlaze koji su delovali „skoro tačno” (kvaternioni su bili gotovo normalizovani, smerovi su otprilike odgovarali, ali su veličine pojedinih komponenti bile dosledno pogrešne, kao da je negde ubačena pogrešna skala). Provera u Vivado simulaciji nije pokazivala ništa neobično jer simulator interpretira VHDL po slovu specifikacije, ali sintetizator je očigledno radio nešto drugačije.

Problem smo lokalizovali u Stage 1 pipeline-a. Izlazi `s1_hgx`, `s1_hgy` i `s1_hgz` (rezultati množenja žiroskopa sa konstantom $\pi/360$ i konverzije iz Q9.7 u Q2.20) bili su registri koji su zatim ulazili u slice operaciju u Stage 2. Vivado je primetio da se `s1_hgx` može apsorbovati direktno u DSP48E1 blok, pri čemu je ignorisao slice operaciju [28 downto 7]. Umesto slajsovanog rezultata u Q2.20 formatu, u Stage 2 je stizao ceo neoslajsovani produkt, što je dovodilo do skaliranja podataka pogrešnim faktorom 64 (2^6). Slice operacija koja je u kodu izgledala kao da uzima bite [28 downto 7] zapravo je u sintezovanom kolu uzimala nešto drugo, što je dovodilo do skaliranja podataka pogrešnim faktorom 16 ili 32.

Rešenje je bilo da dodamo VHDL attribute `keep => "true"` i `dont_touch => "true"` nad tim signalima:

```
attribute keep : string;  
attribute dont_touch : string;  
attribute keep of s1_hgx : signal is "true";  
attribute dont_touch of s1_hgx : signal is "true";
```

Ovi atributi govore sintetizatoru da signal mora ostati netaknut tačno onako kako je deklarisan, što sprečava prebacivanje bitova ili spajanje sa drugim signalima. Nakon ove izmene, simulacija i hardver su počeli da daju iste rezultate.

Vivado optimizator je generalno koristan, ali kada radimo sa fixed-point aritmetikom i preciznim bit-pozicijama, moramo eksplicitno reći alatu šta ne sme da dira. Sintezator nema način da zna da je za naš algoritam tačna bit-pozicija svakog registra suštinska, ne samo njegova funkcionalna vrednost.

10.6 Q-format manipulacija - explicit shift_right + resize

Vezano za prethodno, naučili smo da je za pouzdano upravljanje fixed-point formatima u VHDL-u sigurnije koristiti eksplicitne `shift_right` i `resize` operacije iz `ieee.numeric_std` biblioteke nego direktno slice-ovati bite. U Stage 2 i Stage 4 smo prepisali kod tako da svaka konverzija formata ide kroz:

```
result <= resize(shift_right(signal, N), M);
```

umesto kroz:

```
result <= signal(A downto B);
```

Razlika je suštinska. `shift_right + resize` čuvaju semantiku predznaka i dužine i precizno opisuju matematičku operaciju koja se izvodi. Slice samo uzima sirove bite i daje sintezatoru više slobode da reorganizuje signale na način koji ne želimo. Sa eksplicitnim operacijama, Vivado pravilno reaguje i ne pokušava da prepravlja strukturu signala.

10.7 Debug infrastruktura kroz pipeline

Kada smo pokušavali da lokalizujemo gore opisane probleme sa skaliranjem, suočili smo se sa pitanjem kako uopšte videti šta se dešava unutar hardvera. Sistem je davao samo finalne izlazne kvaternione, a unutrašnji signali pipeline-a nisu bili dostupni za posmatranje na ploči. Da bismo videli šta se dešava u svakoj fazi, dodali smo debug izlazne portove na entitete:

```
dbg_s1_hgx_o  : out signed(21 downto 0); -- iz Stage 1
dbg_s2_adjgx_o : out signed(25 downto 0); -- iz Stage 2
dbg_s3b_dqx_o  : out signed(26 downto 0); -- iz Stage 3b
dbg_s4_shx_o   : out signed(31 downto 0); -- iz Stage 4
```

Ovi signali se propagiraju kroz `ahrs_ip_v3_0` do top-level entiteta i mogu se pratiti ili kroz ILA u Vivadu, ili tako što se BRAM proširi da snima i ove vrednosti za poređenje sa float referencom. Bez ovog mehanizma, debugovanje skaliranja u Stage 1 bi nam verovatno trajalo nedeljama umesto dana. Iako debug signali dodaju malo na utilization (uglavnom kroz IO pinove ili dodatne registre), oni se mogu lako ukloniti u finalnoj verziji.

10.8 Upotreba jednog `fp_mul` IP jezgra

U ranijoj fazi razvoja u Vivado projektu su bila kreirana tri zasebna Xilinx Floating Point IP fajla, `fp_mul`, `fp_mul2` i `fp_mul3`, sa idejom da bi se eventualno mogli konfigurisati sa različitim latencijama radi balansiranja resursa. Ovo je činilo FSM komplikovanim jer je svaka grana morala u principu da računa drugačije čekanje na osnovu toga koja se instanca trenutno koristi.

U finalnoj verziji koda smo se odlučili za jednostavnije rešenje: sve tri paralelne putanje množenja u sekciji 5 instancirane su iz istog `fp_mul` IP jezgra. Konkretno, u arhitekturi `ahrs_sec5` deklarirane su tri komponentna pojavljivanja (`U_MUL`, `U_MUL2` i `U_MUL3`) i sva tri su port-mapom vezana za `fp_mul` (a ne za `fp_mul2` odnosno `fp_mul3`). Nakon sinteze, Vivado u hijerarhiji prikazuje ove tri replike kao `fp_mul`, `fp_mul_1` i `fp_mul_2`, to su Vivado-ova auto-imenovana pojavljivanja istog izvornog jezgra, što se i vidi u tabeli utrošenih resursa u sekciji 7.2.

Posledica: sve tri instance imaju identičnu latenciju (4 takta) i identičan AXI-Stream interfejs, pa je FSM dobio jednu jedinstvenu tabelu tajminga umesto tri različite. Kod je postao mnogo jasniji i lakši za održavanje.

10.9 Verifikacija ispravnosti kroz mixed simulation

Na kraju, još jedna provera ispravnosti bila je mixed simulation kroz Cadence Xcelium gde se VHDL `ahrs_sec678` poziva iz SystemC virtuelne platforme. Tu smo sample-by-sample uporedili izlazne kvaternione sa float referencom iz ESL projekta. Nakon svih prethodnih ispravki, prolazimo svih 1902 sample-a kroz mixed simulation bez assertion failure-a, sa očuvanjem norme kvaterniona unutar tolerancije od oko 0.05% od jedinice kroz ceo dataset. Konkretno, na nekoliko reprezentativnih sample-ova merili smo:

Sample	qw	qx	qy	qz	q
1	+1.0000	-0.0049	-0.0004	-0.0002	1.000000
200	+0.9695	-0.2181	-0.0309	-0.1109	1.000403
600	+0.9890	-0.0400	-0.1300	+0.0500	0.999560
1000	+0.9427	-0.1451	-0.2103	-0.2150	1.000123

Norme su sve unutar 0.05% od 1.0, što potvrđuje da kvaternioni nisu samo numerički stabilni nego i geometrijski ispravni (predstavljaju validne rotacije). Ovo je važna potvrda jer akumulacija fixed-point greške kroz 1092 iteracije kvaternionske integracije lako može dovesti do drift-a u normi, pa činjenica da norma ostaje tako blizu 1.0 znači da algoritam radi ispravno.

11. Literatura

- [1] Materijali sa predavanja i vežbi predmeta Projektovanje složenih digitalnih sistema, Fakultet tehničkih nauka, Univerzitet u Novom Sadu: <https://www.elektronika.ftn.uns.ac.rs/projektovanje-slozenih-digitalnih-sistema/>
- [2] S. O. H. Madgwick, „An efficient orientation filter for inertial and inertial/magnetic sensor arrays”, University of Bristol, Report, 2010.
- [3] Pong P. Chu, „RTL Hardware Design using VHDL: Coding for Efficiency, Portability, and Scalability”, Wiley-Interscience, 2006.
- [4] AMBA AXI4 Protocol Specification, ARM Limited, 2021: <https://developer.arm.com/documentation/ih0022/>
- [5] Zybo Z7-10 Reference Manual, Digilent Inc.: <https://digilent.com/reference/programmable-logic/zybo-z7/reference-manual>
- [6] AXI BRAM Controller v4.1 - LogiCORE IP Product Guide (PG078), AMD/Xilinx, 2019: <https://docs.amd.com/v/u/en-US/pg078-axi-bram-ctrl>
- [7] Floating-Point Operator v7.1 - LogiCORE IP Product Guide (PG060), AMD/Xilinx, 2020: <https://docs.amd.com/v/u/en-US/pg060-floating-point>
- [8] 7 Series DSP48E1 User Guide (UG479), AMD/Xilinx, 2018: https://docs.amd.com/v/u/en-US/ug479_7Series_DSP48E1
- [9] Vivado Design Suite User Guide - Synthesis (UG901), AMD/Xilinx: <https://docs.amd.com/r/en-US/ug901-vivado-synthesis>
- [10] Vitis HLS User Guide (UG1399), AMD/Xilinx - korišćeno za poređenje sa ESL implementacijom: <https://docs.amd.com/r/en-US/ug1399-vitis-hls>
- [11] C. Lomont, „Fast Inverse Square Root”, technical report, 2003 - algoritam korišćen u sekciji 5 za normalizaciju vektora ubrzanja, sa magičnom konstantom 0x5F1F1412 i jednom iteracijom Newton-Raphson korekcije.